

If you used different arrays/structures/classes to represent the different types of sets, would it be possible to have overloaded methods or operations that would provide the correct functionality regardless of whether you were using sets or multisets?

Why or why not? (You do not need to code this, just answer the question).

No, we cannot have an overloaded method that would provide the correct functionality for both sets. We created two classes for each of the types of sets. In each class, we used different data types to represent each set. We used a List to represent a simple set's bit string, and we used a Map to represent a multiset's bit string. Even though some set operations function the same logically, our implementations of the set operations function differently to accommodate each set type representation.

If an overloaded method is allowed to have different code along with different parameters, but retain the same logical functionality, then yes, we could have an overloaded method that functions correctly regardless of the set type.

Would it be possible to use the same data set/structure/class to store sets and multisets? Why or why not?

Yes, it would be possible to use the same class to store sets and multisets. We would be able to use our multisets implementation to store simple sets data because our implementation of multisets stores the number of occurrences of an element alongside of the element. A simple set stored with our implementation of multisets would be a multiset that only has one occurrence of the element. However, we would not be able to store a multiset using our implementation of simple sets. Our implementation of simple sets stores an element only once if it occurs. Data from the multiset would be lost if we used our simple sets implementation.

How easy or difficult is it to determine the type of set that you need to use based on the user's query? Why?

The difficulty of determining which type of set should be used depends on the user's query. If the user is clear about what information they want to have stored in the set and is clear about the requirement for an element to be stored, then it makes it easier to decide what kind of set to use. For example, "Which of our customers attended SLCC?" is a simple query that asks us to store a customer in the set if they attended SLCC. The customer is the element they want to be stored, and the requirement for a customer to be stored is if they attended SLCC.